

WorkTable

Absolutely not a database.

A user's guide to the `worktable!` macro, its queries, its indexes and its persistence tier. Written against 1.9.0-alpha1.

What this is

Embedded table storage for Rust. Declare a table with a macro, get a typed struct back: a primary key, secondary indexes, generated queries. Rows live in memory as paged, zero-copy records. Persisting them to local disk or S3 is opt-in.

What it is not. No transaction journal, no fsync per batch. A mutation returning means the change was accepted and queued, not that it is on stable storage. See [Persistence](#).

Getting started

```
cargo add worktable
```

```
use worktable::prelude::*;
use worktable::worktable;
```

Everything the macro emits resolves through `worktable::prelude`, so that one import is the whole setup.

Examples

Every clause the macro accepts appears below, labelled where it is used.

1. The smallest table

```
worktable! (
    name: Order,                // required, and must come first. CamelCase.
    columns: {
        id: u64 primary_key,    // exactly one primary key is required
        total: u64,
    },
);

let table = OrderWorkTable::default();
table.insert(OrderRow { id: 1, total: 500 }); // errors if the key exists
table.upsert(OrderRow { id: 1, total: 600 }); // overwrites instead
let row = table.select(1).expect("just inserted");
```

`name: Order` generates `OrderWorkTable`, `OrderRow`, `OrderPrimaryKey`, and for a persisted table `OrderPersistenceEngine`.

2. Column clauses

```
worktable! (  
  name: Account,  
  columns: {  
    id: u64 primary_key autoincrement, // the table assigns keys  
    email: String, // any sized type  
    nickname: String optional, // becomes Option<String>  
    balance: i64,  
  },  
);
```

Clause order inside a column is fixed by the grammar and is not the order you might guess: `<name>: <Type> [primary_key [autoincrement|custom]] [optional] [columnar(..)]`. The generator binds to `primary_key`, and `optional` follows both.

```
let id = table.insert(AccountRow {  
  id: 0, // ignored under autoincrement  
  email: "a@b.c".to_string(),  
  nickname: None, // optional column  
  balance: 0,  
})?;
```

`custom` replaces `autoincrement` when you generate keys yourself and still want the table to track the high-water mark.

3. Composite primary keys

```
worktable! (  
  name: Quote,  
  columns: {  
    exchange: u32 primary_key, // both columns carry primary_key  
    symbol: u32 primary_key, // one generator is shared between them  
    price: f64,  
  },  
);  
  
let row = table.select((1_u32, 42_u32).into()).expect("present");
```

A composite key keeps `worktables_index` even though the default is `arctic`, because `arctic` cannot represent a tuple key.

4. Secondary indexes

```
worktable! (
  name: Customer,
  columns: {
    id: u64 primary_key,
    email: String,
    country: u16,
  },
  indexes: {
    // <name>: <column> [unique] [using <backend>]
    email_idx: email unique using worktables_index, // one row back
    country_idx: country using arctic,               // many rows back
  },
);

let one = table.select_by_email("a@b.c".to_string()); // Option<Row>
let many = table.select_by_country(44).execute()?;    // Vec<Row>
```

`using` is optional and defaults to `arctic`. An index over an optional or variable-width column must say `using worktables_index`; `arctic` cannot key one.

5. Declared queries

```
worktable! (
  name: Invoice,
  columns: {
    id: u64 primary_key,
    amount: u64,
    state: u8,
  },
  queries: {
    update: {
      AmountById(amount) by id, // <Name>(<columns>) by <key>
    },
    delete: {
      ById() by id, // empty parens: names no columns
    },
    in_place: {
      StateById(state) by id, // only `by <primary key>` is supported
    },
  },
);
```

CamelCase declared, snake_case generated:

```
table.update_amount_by_id(AmountByIdQuery { amount: 900 }, 1).await?; // name + "Query"
table.delete_by_id(1).await?;
table.update_state_by_id_in_place(1, |state| *state = 2).await?;
```

`update` reads, changes and writes. `in_place` mutates without selecting first and locks internally, so it is safe from several threads without the caller holding anything.

6. Selects you do not declare

Generated from the columns and indexes, so none of these appear in the macro:

```

table.select(id) // primary key
table.select_by_email("a@b.c".to_string()) // unique index
table.select_by_country(44).execute()? // non-unique index
table.select_by_pk_range(10..=20).execute()? // range over the primary key
table.select_by_country_range(40..=50).execute()? // range over an indexed column
table.select_all().execute()?
table.select_all()
    .order_on(InvoiceRowFields::Amount, Order::Desc) // generated field enum
    .limit(10)
    .execute()?

```

7. Columnar fields and indexes

```

worktable! (
  name: Reading,
  columns: {
    id: u64 primary_key, // must NOT say columnar: implicit already
    host_id: u64 columnar(chunk_rows(2), compression(none)),
    timestamp: i64 columnar, // bare form, not the same as columnar(..)
    payload: String, // row-wise only
  },
  columnar_indexes: {
    host_time: { // <name>: { cluster_by: [...] }
      cluster_by: [host_id, timestamp], // every field must be columnar
    },
  },
  config: {
    columnar_slot_id: ColumnSlotId16, // slot width, default ColumnSlotId32
    columnar_chunk_rows: 4096, // default chunk size, default 65536
  },
);

```

A `columnar` column is stored column-wise as well as row-wise, so a scan over that field reads only that field's bytes.

8. Page size and row derives

```

worktable! (
  name: Small,
  columns: { id: u64 primary_key, v: u64 },
  config: {
    page_size: 4096, // 512 minimum, 65535 max under arctic
    row_derives: Clone, Debug, // bare identifiers, NOT [Clone, Debug]
  }, // row_derives must be written last
);

```

`row_derives` reads identifiers until it meets another config key, which is why it goes last. The `config` block takes no trailing comma after its closing brace.

9. Partitioned tables

```
worktable! (
  name: Book,
  persist: false,
  partition_by: symbol_id: u16, // <name>: <unsigned type>, stored per partition
  columns: {
    exchange_id: u8 primary_key,
    bid: f64,
    ask: f64,
  },
);
```

The partition key is stored once per partition rather than once per row, and no query can name it.

10. Choosing a runtime

```
worktable! (
  name: Orders,
  runtime: nagoya(shared_slot), // or `tokio`, which takes no flavor
  columns: { id: u64 primary_key, total: u64 },
);
```

Omitting `runtime:` and writing `runtime: nagoya(shared_slot)` describe the same table. A per-query-block form parses but codegen ignores it today:

```
queries: {
  update runtime fast_local: { // parses, currently has no effect
    TotalById(total) by id,
  },
},
```

11. A persisted table, end to end

```
worktable! (
  name: Ledger,
  version: 2, // optional, defaults to 1. Must precede persist.
  persist: true, // generates LedgerPersistenceEngine
  columns: {
    id: u64 primary_key,
    amount: i64,
  },
);

let config = DiskConfig::new_with_table_name(
  dir,
  LedgerWorkTable::name_snake_case(),
  LedgerWorkTable::version(),
);
let engine = LedgerPersistenceEngine::new(config).await?;
let table = LedgerWorkTable::load(engine).await?; // replays what is on disk

table.upsert(LedgerRow { id: 1, amount: 42 }).await?; // queued, not durable

table.close().await?; // the only thing that proves the queue drained
```

12. Everything at once

The prefix is ordered. Everything after `partition_by` is free-order.

```
worktable! (  
  name: Kitchen,                // 1, required  
  version: 3,                   // 2, optional  
  persist: false,               // 3, optional  
  partition_by: shard: u16,     // 4, optional  
  runtime: nagoya(locality),    // free-order from here down  
  columns: {  
    id: u64 primary_key autoincrement,  
    nickname: String optional,  
    bucket: u32 columnar,  
    score: i64,  
  },  
  indexes: {  
    nickname_idx: nickname unique using worktables_index,  
    score_idx: score,  
  },  
  columnar_indexes: {  
    by_bucket: { cluster_by: [bucket] },  
  },  
  queries: {  
    update: { ScoreById(score) by id },  
    delete: { ById() by id },  
    in_place: { ScoreById(score) by id },  
  },  
  config: {  
    page_size: 4096,  
    columnar_chunk_rows: 4096,  
    row_derives: Clone, Debug,  
  },  
);
```

Writing ``version`` or ``persist`` late. The prefix keys are positional and the error says so rather than reporting an unexpected token. `version` after `columns` is refused; so is `persist` or `partition_by`.

Index backends

`using` names the physical structure. They differ in what they can express, not only in speed.

Backend	When it fits
<code>worktables_index</code>	The general one. Takes an ordered key of any type, and the only one that can key an optional or variable-width column.
<code>indexset</code>	Vanilla IndexSet, selectable explicitly while keeping the same disk representation.
<code>arctic</code>	The default. Fixed-width keys only, and the fast one. Packs a row link into a single <code>u64</code> .
<code>congee</code>	Fixed-width integer keys. Refuses <code>String</code> and other variable-width types.

Rules:

- Omitting `using` gives `arctic`. A composite primary key keeps `worktables_index`, because `arctic` cannot represent a tuple key.

- Congee must state `persist` explicitly. Its persistence uses native checkpoint and WAL adapters rather than the shared page format.
- Arctic cannot key an optional or variable-width column. `nickname_idx: nickname unique` over a `String optional` is rejected, and the message names the type rather than the omission. Say using `worktables_index`.
- Arctic caps page size at 65535: it packs a link into 64 bits with 16-bit offset and length fields. The macro refuses the combination.

Page size

A page has two sizes and they are not interchangeable. The **stride** is what one page occupies on disk, header included, and every file offset is computed from it. The **inner size** is the stride less the 28-byte header: what a page can actually hold.

Set it in the `config` block:

```
worktable! (
  name: Small,
  columns: { id: u64 primary_key, v: u64 },
  config: { page_size: 4096 }
);
```

Persisted tables have a floor, not a fixed size. `page_size` works for a persisted table, and the only rule is a 512-byte minimum: a page on disk carries a 28-byte header, so anything much smaller is mostly header. An Arctic-backed table is also capped at 65535. In-memory tables have neither limit.

It was refused outright until recently, because the seeks computed offsets from a hardcoded constant while the table threaded the configured one. Every location that decides a page size, and the three silent bugs found while making them agree, are in `docs/page-size.md`.

Columnar rules

Syntax is in [Example 7](#). The constraints:

- Every field in `cluster_by` must itself declare `columnar`.
- A primary-key column must not declare `columnar`. It participates in columnar identity implicitly, and declaring it again generates duplicate scan methods.
- `columnar_indexes` requires at least one `columnar` field.
- A columnar index must not take the name of a columnar field, which would generate two scan methods with one name.
- `columnar_slot_id` and `columnar_chunk_rows` live in `config` because they apply to the table. Defaults are `ColumnSlotId32` and 65,536.

Persistence

Persistence is implemented, not planned. Add `persist: true` and load the table through an engine.

```
let config = DiskConfig::new_with_table_name(dir, OrderWorkTable::name_snake_case(),
OrderWorkTable::version());
let engine = OrderPersistenceEngine::new(config).await?;
let table = OrderWorkTable::load(engine).await?;
```

S3 layers on top of the disk engine rather than replacing it: `S3SyncDiskPersistenceEngine` wraps a `DiskPersistenceEngine` and syncs it. Enable the `s3-support` feature.

The durability contract

Boundary	What you actually get
A mutation returns	The in-memory change was accepted and its persistence operation was queued.
<code>wait_for_ops()</code> returns	The engine completed the queued operations. No fsync, no stable-storage guarantee.
<code>close()</code> returns	Intake stopped, the queue drained, the engine task joined. Still no fsync guarantee.
Process crash or <code>SIGKILL</code>	Acknowledged rows may be lost and the file may be torn.
Power loss	No atomic-batch or stable-storage guarantee.

Call `close()` on orderly shutdown. `wait_for_ops()` is not a shutdown boundary: it does not stop another task queueing more work, so it means nothing without writer quiescence.

Persistence failure is terminal. An event gap, queue-analysis error, batch-apply error or engine-task failure fails the table, and the original error goes to waiters, to `close()` and to later mutations.

Loading a torn store

A normal load audits archived rows and both primary and secondary index consistency before exposing the table, and refuses torn state with `PersistenceLoadError` rather than opening plausible-but-invented rows.

`LoadMode::Recovery` exists for offline tools only. It copies individually validated rows through a surviving index into a clean table, which must then pass a normal strict load before anyone reads it. It is not an in-place repair and must never serve live traffic.

Vacuum

Persisted vacuum compacts the in-memory layout and keeps disk indexes consistent with moved rows. It does not truncate `.wt.data`. Watch physical growth with `persisted_data_file_size_bytes().await` and decide when to snapshot and rebuild.

The filesystem

One module, `worktable::prelude::fsx`, naming no async runtime. The file type is `std::fs::File` behind `AllowStdIo`: blocking semantics, `futures-io` traits.

Measured, not assumed. `tokio::fs` ran scattered updates at 12,316 rows per second against 74,728 on `std::fs`, a factor of 6.1, with bulk insert within noise. A scattered update is many small IOs and `tokio::fs` pays a thread-pool round trip for each.

Where to put the work. The calls block, so a persistence engine should own a thread rather than share a worker pool. They were never waiting on the disk anyway: 89 voluntary context switches across 25,000 inserts.

Choosing a runtime

Syntax is in [Example 10](#). The parenthesised name is a **flavor**: a set of scheduler tunings, not a different scheduler. All flavors share one pool, so choosing between them costs no extra code and no rebuild.

Flavor	What it changes
<code>shared_slot</code>	The default. Keeps a self-waking task on its worker, and shares the overflow when more than one piles up behind it.
<code>locality</code>	Keeps a self-waking task on its worker and never shares the overflow.
<code>spread</code>	Sends every wake through the shared injector instead of keeping it local.
<code>throughput</code>	<code>spread</code> , taking a larger batch from the injector at a time.
<code>wide_injector</code>	<code>spread</code> , taking a larger batch still.
<code>low_latency</code>	<code>locality</code> , looking for work more often before parking.

Take the default. Measured across a read/write mix, YCSB and a persisted mix, every flavor lands inside the run-to-run noise of every other, on 9 to 16 repetitions per point. The one choice that changes anything is a negative: putting an injector-waking flavor (`spread`, `throughput`, `wide_injector`) on a write-heavy table costs 55% to 57%, because the workload wakes on every await. The default does not do that.

Not a knob to tune per table. If you do measure, report a range rather than a median: a 3-run reading of this reversed twice under 16 runs.

Concurrency

Indexes are lock-free with change-data-capture; a row-level `LockMap` gives ordered access when you want it. Reads always use immutable row-version publication, including under `default-features = false`: turning off a feature must never expose a safe API that races deserialization against page-byte mutation.

Point lookups use a strict backend-specific visibility contract. `WorkTablesIndex` pins the structural mapping until its node is locked, so hits and misses are both definitive.

Feature flags worth knowing

Feature	Effect
<code>std</code>	On by default. Off, the crate links no <code>std</code> and the whole persistence half is gone with it.
<code>s3-support</code>	The S3 sync engine, and the HTTP stack under it.
<code>logical-index-persistence</code>	Moves unique structural CDC work off the mutation path into the background worker. The page format is unchanged either way.
<code>wti-predictable-search</code>	On by default. The branch-based node search, which avoids a measured regression on sequential numeric keys.

The three alternative search policies (`wti-hybrid-search`, `wti-std-search`, `wti-superslice-search`) are compile-time gates. Enable one, and only one, for an unambiguous build. If feature unification turns on several, `WorkTablesIndex` applies a documented precedence rather than refusing the graph.

Where to look next

Document	Covers
----------	--------

<code>docs/persistence-durability.md</code>	The durability contract in full, and the snapshot-restore procedure.
<code>docs/index-backend-dsl-proposal.md</code>	The <code>using</code> syntax and the capability matrix per backend.
<code>docs/page-size.md</code>	Every location across the four crates that decides a page size.
<code>docs/queries.md</code>	The generated query surface and the custom query grammar.
<code>docs/migration.md</code>	Moving a store between formats.
<code>docs/known-issues.md</code>	What is known to be wrong right now.